



Fundamentos da Programação Orientada a Objetos

Tratamento de Exceção

Nesta unidade, abordaremos a manipulação e o tratamento de erros. Entretanto, não estamos nos referindo a achar erros de código, o que a Integrated Development Environment (IDE) poderia encontrar facilmente. Mesmo quando programamos profissionalmente e dominamos a linguagem, há várias situações legítimas em que problemas podem ocorrer. Em outras palavras, **nem sempre um erro é capturado pelo compilador antes de você testar o código.**

Além disso, o ideal também é sermos **preventivos**, para que um erro de lógica seja imediatamente identificado e corrigido. No Java, esse tratamento é realizado por meio do **mecanismo de exceções**, que pode ser usado para:

+ Proteger

Quando escrevemos nossas classes, podemos criar e disparar nossos próprios erros para evitar que uma regra de negócio seja violada. Por exemplo, vamos supor que criemos uma classe Aluno. Certamente, iremos querer impedir que alguém cadastre um valor negativo no campo da idade ou da matrícula, e podemos fazer isso disparando uma exceção.

+ Tratar

Algumas situações de erro são conhecidas, mas podem ser identificadas e tratadas de maneira elegante em um sistema. Por exemplo, digamos que em uma aplicação de edição de textos, o usuário tente criar um arquivo em uma pasta em que ele não possui acesso. O Java disparará um erro, mas sua aplicação pode identificar essa ocorrência e,

em vez de interromper seu funcionamento, ela irá mostrar uma mensagem de erro ao usuário e simplesmente pedir para que ele escolha outro local.

+ Registrar

Infelizmente, algumas vezes erros completamente imprevistos ocorrerão. Neste caso, pode ser interessante tratá-los de alguma forma, como registrá-los em um arquivo, para posterior análise.

+ Gerenciar recursos

Podemos utilizar o mecanismo de exceções para garantir que recursos utilizados pela aplicação sejam corretamente finalizados. Por exemplo, pode ser necessário fechar um arquivo aberto, ou encerrar uma conexão de redes.

Tudo isso adicionará um grau adicional de robustez em nossas aplicações, tornando nosso código defensivo e evitando que problemas ainda maiores se propaguem ao longo do tempo. E sabe quem gosta de desenvolvedores que criam algoritmos robustos? Exatamente: o mercado.

Vamos lá?

| Lidando com as mensagens de erro



EXEMPLO

Exemplo de aplicação 1: crie um projeto em Java com uma classe chamada **exemplo**. Cole o código a seguir, e o teste.

```
</>
```

```
1 public class Exemplo {  
2     static void realizarConta() {  
3         int x = 0;  
4         int y = 10;  
5         int z = y/x;  
6         System.out.println(z);  
7     }  
8  
9     public static void main(String[] args) {  
10        realizarConta();  
11    }  
12 }
```

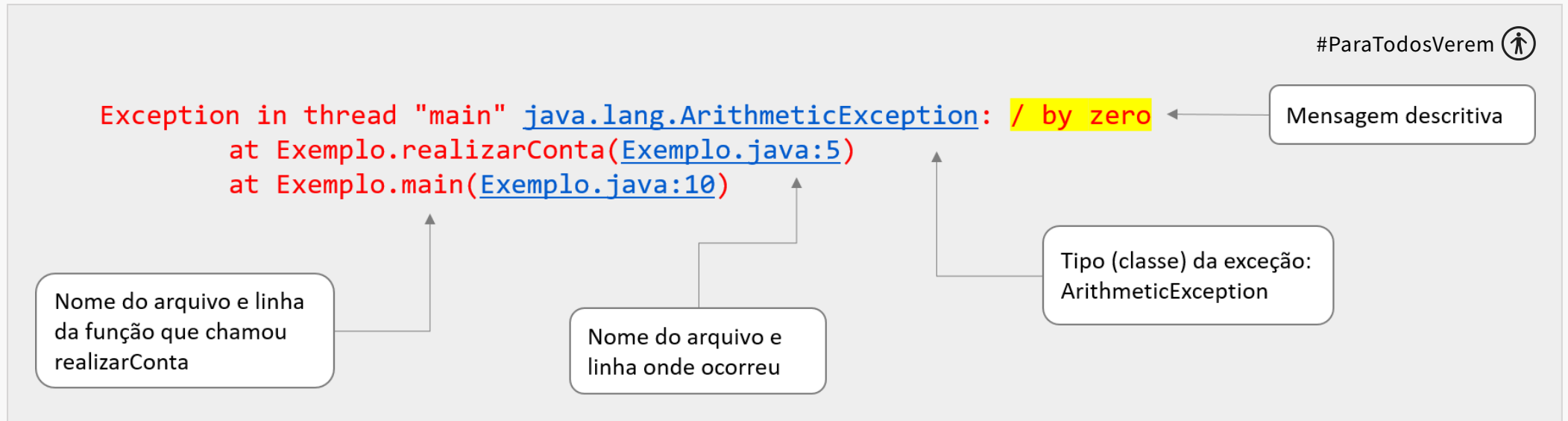
Você pode ter percebido que o código deu um erro **depois que você executou** o projeto. Isto é: ao contrário de muitos casos que vimos até o momento em que o erro antes de executar o código, agora somente soubemos do erro após a sua execução.

Perceba que acontece um erro na linha 5. O erro que apareceu foi algo parecido com isto:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero  
    at Exemplo.realizarConta(Exemplo.java:5)  
    at Exemplo.main(Exemplo.java:10)
```

Agora, **por que apareceu este erro?** A mensagem “java.lang.ArithmeticException: / by zero” é autoexplicativa: há um erro de divisão por zero. Este exercício, apesar de simples, nos mostra um exemplo do uso do mecanismo de exceção. Perceba que o código em questão possui um erro, uma vez que tem duas variáveis inteiras, sendo a variável **x** com valor 0. A divisão de **y** por **x** da linha 5 é, portanto, impossível. E como o Java

sinaliza esse problema? Ele irá disparar uma **exceção**. Como nesse código não fizemos qualquer tratamento especial desse problema, o método main foi encerrado e o Java imprimiu em sua saída de erro (**System.err**) a descrição do erro, conhecida como **stack trace**. A figura a seguir ajudará você a entender em detalhes essa mensagem.



Fonte: Autores (2021).

Aqui, já nos deparamos com uma excelente característica do mecanismo: os erros são extremamente descritivos (apesar de não parecer a primeiro momento). Observe que:

1. O erro possui em sua primeira linha uma **mensagem descritiva**, que explica o problema ocorrido.
2. Além da mensagem, o erro é um objeto e, portanto, possui uma classe. Essa classe normalmente possuirá um nome que permite-nos entender o erro quanto a sua natureza. Neste caso, trata-se de uma *ArithmeticException*, uma vez que foi uma operação aritmética que causou o problema.
3. Por fim, há o **stack trace** propriamente dito. Trata-se da descrição do ponto exato do código em que o erro ocorreu. O Java inicia pelo ponto mais específico e lista, linha a linha, todos os arquivos e métodos que foram chamados para que o código chegasse até ali. Por isso, temos duas linhas descritas na mensagem: a da função `realizarConta` e a do próprio método `main`. Isso permite que rastreemos não só o

ponto em que o erro ocorreu, mas tenhamos uma boa noção do seu contexto. Afinal, um mesmo método pode ser chamado de diferentes locais no código.



DICA

Não se assuste com o tamanho do **texto da exceção**, pois é perfeitamente normal que ela atinja várias linhas. Aprenda a ler o stacktrace e use-o a seu favor.

| Tratamento de erros

Pode parecer bastante fácil corrigir esse código: basta informar um valor correto para `y`, não é? Agora, o que poderíamos fazer caso `y` não fosse diretamente definido, mas, sim, lido do teclado? Neste caso, o erro poderia ocorrer apenas algumas vezes, quando nosso usuário digitasse o valor 0. Vamos explorar algumas formas diferentes para realizar esse tratamento.

| Utilizando condicionais

A primeira forma que possuímos para tratar erros é utilizar condicionais que testem o problema antes que ele ocorra, por exemplo:

```
</>
```

```
1 import java.util.Scanner;
2
3 public class Exemplo {
4     static void realizarConta(int x, int y) {
5         if (x == 0) {
6             System.out.println("Divisão por zero");
7         } else {
8             System.out.println(y/x);
9         }
10    }
11
12    public static void main(String[] args) {
13        Scanner sc = new Scanner(System.in);
14
15        System.out.println("Digite o valor de x: ");
16        int x = sc.nextInt();
17        System.out.println("Digite o valor de y: ");
18        int y = sc.nextInt();
19    }
```

Essa abordagem, porém, apresenta um grande problema. Nem sempre a função que trata o erro é a mesma função que realiza o cálculo. Por exemplo, poderíamos querer manter a função `realizarConta()` somente com o cálculo e deixar no `main` a impressão do texto. Em outras palavras, a divisão das duas variáveis e a chamada do `System.out.println()` poderiam ocorrer em momentos diferentes do código. Nesse caso, o ideal é **capturar a exceção** gerada pela divisão.

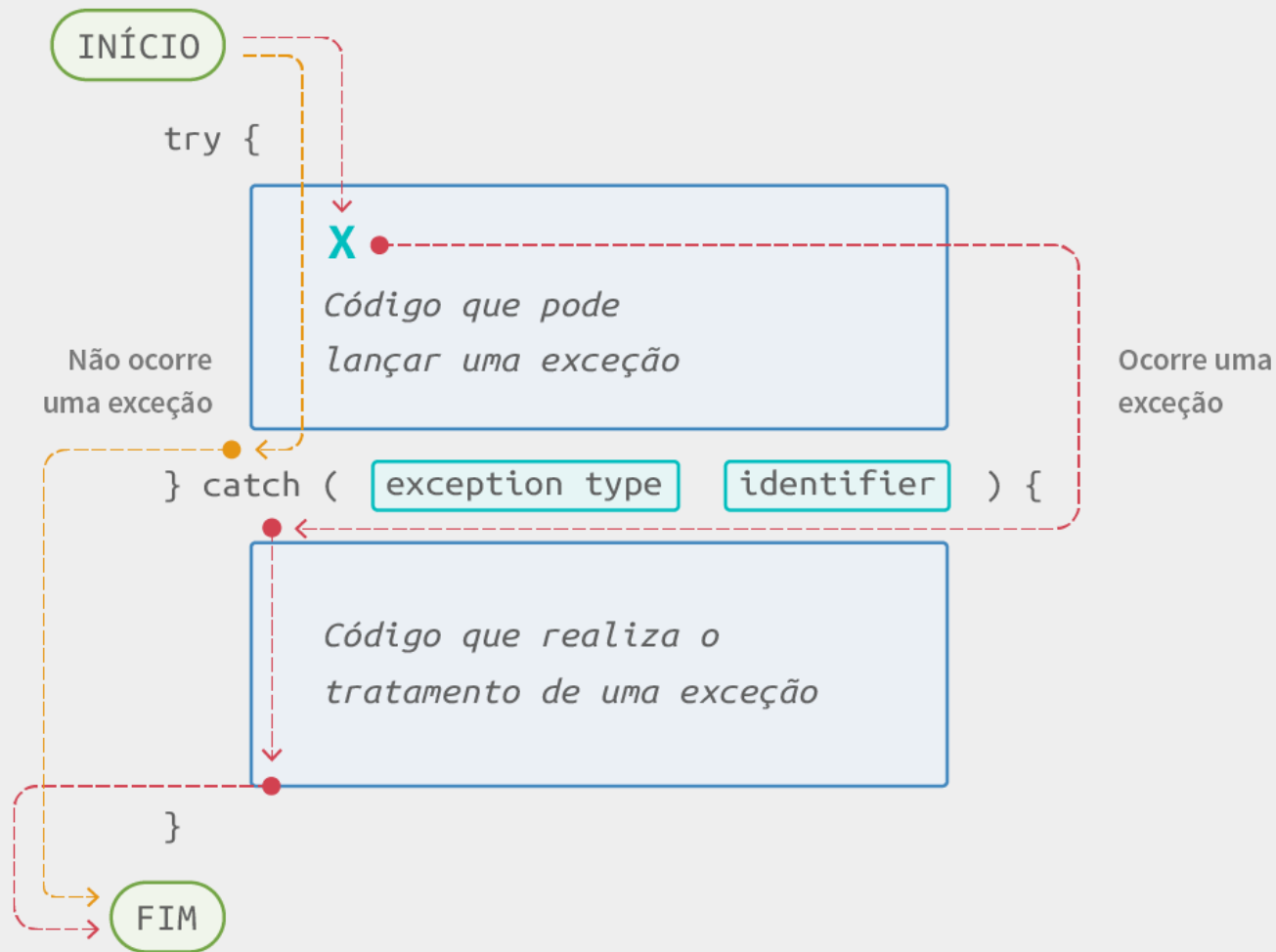
Capturando exceções

Quando uma exceção ocorre, ela abandona imediatamente o método que a gerou. Em seguida, ela é propagada para a função que chamou esse método e, caso haja um tratamento, essa propagação é interrompida. Caso não haja, ela fará com que esse método também seja abandonado, e esse ciclo pode continuar até que ela deixe o *main*, finalizando o programa e encerrando a aplicação.

Utilizamos o comando ***try...catch*** para fazer esse tratamento. Primeiramente, vamos verificar como esse esquema de ***try*** (**tente**) e ***catch*** (**capture**) funciona, conforme a figura a seguir.

Observe os blocos de comandos *try* e *catch*:

- Se ocorrer uma exceção dentro do bloco *try*, o código é desviado para o bloco *catch* correspondente.
- Pode haver mais de um bloco *catch*, cada um pegando uma **exceção diferente** e realizando o código que **trata dessa exceção**.
- Para saber qual desvio ou bloco *catch* deve ser executado, precisamos identificar a **classe da exceção** e nomear um **objeto da exceção**, conforme indicado na figura.
- Se não ocorrer uma exceção dentro do bloco *try*, a execução **pula** os blocos de *catch* e finaliza.



Fonte: Autores (2021).

Agora, vamos verificar como esse esquema é implementado, conforme apresentado no exemplo a seguir.



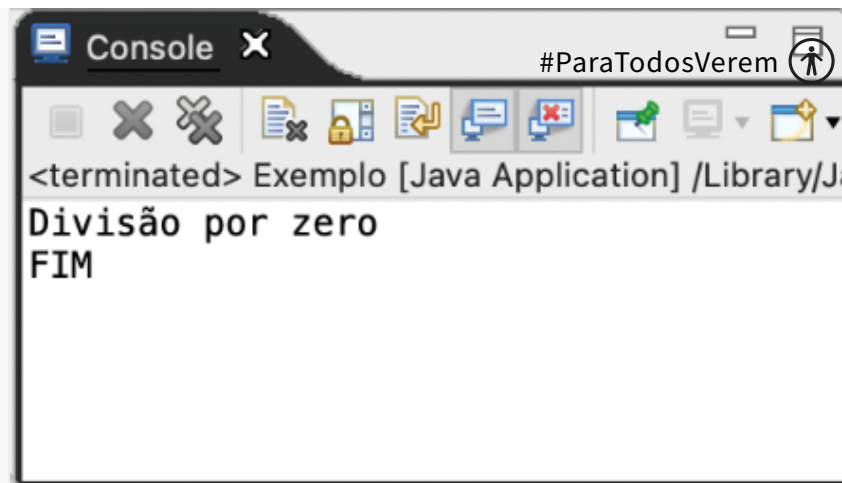
EXEMPLO

Exemplo de aplicação 2: adapte o código do exemplo anterior para incluir tratamento de exceções.

O código modificado deverá ficar parecido com este – veja a inclusão de try/catch nas linhas 7, 10, 11 e 12. Observe que a classe da exceção é a **ArithmeticException** (erros aritméticos como, por exemplo, erros de divisão por zero) e o objeto da classe da exceção se chama “e” (linha 10), neste exemplo.

</>

```
1 public class Exemplo {
2     static int realizarConta(int x, int y) {
3         return y / x;
4     }
5
6     public static void main(String[] args) {
7         try {
8             int z = realizarConta(0, 10);
9             System.out.println(z);
10        } catch (ArithmeticException e) {
11            System.out.println("Divisão por zero");
12        }
13        System.out.println("FIM");
14    }
15 }
```



Teste o código. A mensagem “Divisão por zero” aparecerá quando o valor da variável **y** é igual a zero. Tente mudar o valor de y: o código deverá mostrar o resultado da divisão normalmente.

Iniciamos o método main com o comando try. Em seu interior, inserimos todo o código que poderia disparar um erro. Caso um erro ocorra, ele será imediatamente disparado para o comando catch da **linha 10**. A variável e, do tipo ArithmeticException, irá conter a descrição do erro gerado. Como a exceção foi capturada e tratada, o programa irá então interromper sua propagação e **continuar para a linha após o catch**. É por isso que, mesmo com o erro, o texto “FIM” da **linha 13** ainda é impresso.

E se nenhum erro ocorrer? O programa executará o println da **linha 9** e então pulará o bloco catch, executando também a **linha 13**.

Múltiplos blocos de catch

Vimos que dentro do bloco try, podemos colocar mais de uma linha de código. Mas e se nessas linhas, exceções diferentes forem disparadas? Podemos utilizar múltiplos blocos de catch, conforme demonstrado a seguir.



EXEMPLO

Exemplo de aplicação 3: adapte o código do exemplo anterior para tratar diferentes tipos de exceções.

Teste o código abaixo, substituindo o que já implementamos até o momento. Confira se você consegue gerar a exceção `ArithmeticException`.

</>

```
1 public class Exemplo {
2     static int realizarConta(int x, int y) {
3         return y / x;
4     }
5
6     public static void main(String[] args) {
7         try {
8             int z = realizarConta(2, 10);
9             System.out.println(z);
10
11             String x = null;
12             System.out.println(x.length());
13         } catch (ArithmeticException e) {
14             System.out.println("Divisão por zero");
15         } catch (NullPointerException e) {
16             System.out.println(e.getMessage());
17         }
18         System.out.println("FIM");
19     }
```

Observe que o código só é desviado para o catch em que a exceção ocorreu. Os demais blocos de catch não serão executados. Aqui, o código irá para a linha 14 e mostrará a mensagem “divisão por zero” somente se houve alguma exceção do tipo `ArithmeticException` entre as linhas 8 a 12. Agora, se houve algum tipo de erro de valores nulos entre as linhas 8 a 12, então teremos uma exceção do tipo `NullPointerException` e iremos

para a linha 16.

Além disso, observe que dessa vez utilizamos o objeto “e” recebido por parâmetro. Ele permite acessar dados da exceção, como sua mensagem. Esta mensagem é a mesma que sairia no **stack trace**, caso deixássemos a exceção sair do bloco main (tente testar isso você mesmo).



DICA

Além do getMessage(), você pode mandar imprimir o **stack trace** completo, tal como ocorre quando o código sai do *main*, com o comando: `e.printStackTrace()`.

Multi-catch

Muitas vezes, um grupo de exceções apresenta tratamento idêntico, tal como mostrar a mensagem de erro, ou ignorá-la para que o usuário possa repetir a ação. Nesse caso, você pode especificar um grupo de exceções a serem capturadas por meio do operador de “|” (chamado de pipe), como apresentado no exemplo a seguir.



EXEMPLO

Exemplo de aplicação 4: adapte o código do exemplo anterior para incluir o uso de **multi-catch**.

Execute o programa a seguir e confira se você consegue gerar a exceção `ArithmeticException`.

```
</>
```

```
1 public class Exemplo {
2     static int realizarConta(int x, int y) {
3         return y / x;
4     }
5
6     public static void main(String[] args) {
7         try {
8             int z = realizarConta(2, 10);
9             System.out.println(z);
10
11             String x = null;
12             System.out.println(x.length());
13         } catch (ArithmeticException | NullPointerException e) { // 2 tipos de
14             System.out.println(e.getMessage());                // exceção
15                                                                // tratados da
16             }                                                    // mesma forma
17         System.out.println("FIM");
18     }
19 }
```



DICA

A linha 13 estabelece que qualquer uma dessas duas exceções serão tratadas pela linha 14. Em Java, o operador de pipe ("|") é utilizado para realizar operações lógicas de "OU" bit a bit entre dois valores inteiros.

Por fim, é importante saber que as exceções também fazem parte de uma **hierarquia de classes**. Isso torna possível capturá-las de maneira hierárquica, como se em cada catch houvesse um teste de instanceof. Todas as exceções derivam da classe exception, o que nos permitirá reescrever o catch como:

```
} catch (Exception e) {  
    System.out.println(e.getMessage());  
}
```

O código apresentado captura **todos os tipos de exceções possíveis**.

É possível até mesmo combinar as três abordagens, desde que as regras mais abrangentes fiquem por último e as mais específicas, por primeiro. O *try* entrará no primeiro catch que encontrar, sem executar os demais. Discutiremos mais sobre **exceções hierárquicas** e sua importância em breve, no tópico **disparando exceções**.

Cláusula *finally*

Em algumas situações, podemos querer que um trecho de código seja sempre executado, independentemente de uma exceção ter disso disparada ou não. Para esses casos, podemos associar ao *try* a cláusula ***finally***. Veja um exemplo a seguir.



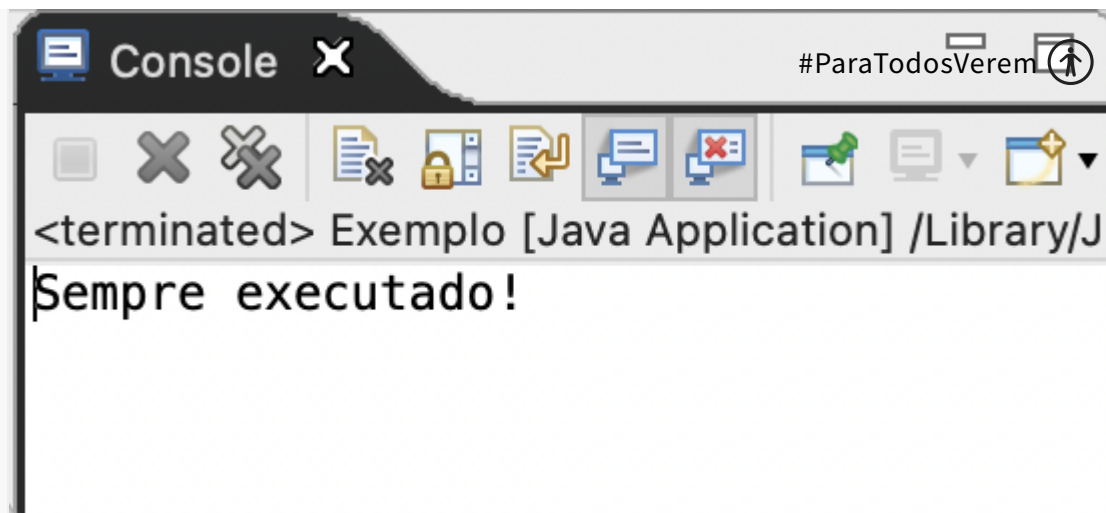
EXEMPLO

Exemplo de aplicação 5: adapte o código do exemplo anterior para incluir o uso de ***finally***.

Execute o programa a seguir e confira se você consegue apresentar a mensagem, conforme tela apresentada.

</>

```
1 public class Exemplo {
2     static int realizarConta(int x, int y) {
3         return y / x;
4     }
5
6     public static void main(String[] args) {
7         try {
8             int z = realizarConta(2, 10);
9             if (z == 5) return;
10
11             System.out.println(z);
12
13             String x = null;
14             System.out.println(x.length());
15         } catch (Exception e) {
16             System.out.println(e.getMessage());
17         } finally {
18             System.out.println("Sempre executado!");
19         }
```



DICA

*Em sistemas reais, evite capturar uma **Exception** genérica. Capture apenas o que você sabe tratar (como **ArithmeticException** para erros de aritmética). Capturar tudo pode esconder bugs graves no seu código*

Note que o código dentro do *finally* foi executado mesmo com a presença do *return* da **linha 9**! Como exercício, teste outros valores para o método realizar conta e observe como o *finally* sempre será executado – não importando se uma exceção foi disparada, se houve um *return*, ou mesmo, se o método executou até o final.

Por fim, um último detalhe: blocos *try...finally*, sem nenhum *catch*, também são permitidos dentro do Java. Isso significa que você não precisa usar sempre a combinação *try/finally/catch*, mas *try/finally* ou *try/catch* também são combinações aceitas em Java.

Vimos anteriormente que a cláusula *finally* é muito utilizada para fechar recursos, como conexões de banco de dados, arquivos abertos ou objetos Scanner.

Entretanto, ter que escrever um bloco *finally* apenas para chamar o método `.close()` pode tornar o código repetitivo e verboso. Além disso, se esquecermos de fechar um recurso, podemos causar problemas de performance no computador (vazamento de memória).

Para resolver isso de forma elegante, o Java introduziu o **try-with-resources** (ou “tentar com recursos”). A ideia é simples: você declara e instancia o recurso **dentro dos parênteses** do comando *try*. Ao fazer isso, o Java se encarrega de fechar esse recurso automaticamente assim que o bloco terminar, seja com sucesso ou com erro.

Não é necessário escrever o *finally* e nem chamar o `close()` manualmente. O que acha de adaptarmos o exemplo de aplicação 5 para este novo formato? Aqui, vamos adaptar o código de cálculo para ler os números do teclado. Observe como o Scanner é gerenciado automaticamente, mesmo com múltiplos pontos de saída no código.

</>

```
1 import java.util.Scanner;
2
3 public class Exemplo {
4     static int realizarConta(int x, int y) {
5         return y / x;
6     }
7
8     public static void main(String[] args) {
9         // O recurso (Scanner) é declarado dentro dos parênteses do try
10        try (Scanner sc = new Scanner(System.in)) {
11
12            System.out.print("Digite o divisor (x): ");
13            int x = sc.nextInt();
14
15            System.out.print("Digite o dividendo (y): ");
16            int y = sc.nextInt();
17
18            int z = realizarConta(x, y);
19
```

Agora, o que está acontecendo aqui? Vejamos:

1. **Linha 10:** veja que na mesma linha do try (...) abrimos o Scanner. Com isso, o Java entende que ele deve ser encerrado ao final.
2. O recurso (no nosso caso, o Scanner) será fechado automaticamente em três situações distintas aqui:
 - **Sucesso:** se o código rodar até o fim do bloco try.
 - **Interrupção (return):** se a condição `z == 5` for verdadeira, o Java fecha o Scanner antes de executar o return.
 - **Exceção:** se ocorrer divisão por zero (na leitura dos números) ou o `NullPointerException` (na linha do String `s`), o Java fecha o Scanner antes de passar o controle para o bloco catch.



DICA

Use sempre o try-with-resources para conexões de banco, arquivos e scanners. Isso garante que você nunca deixará "portas abertas" na memória do servidor, mesmo que sua lógica de programação tenha saídas abruptas.

| Criando e disparando exceções

O mecanismo de exceções também permite que você crie e dispare suas próprias exceções. Além disso, ele fornece um conjunto de classes padrão de exceção, que facilitam o trabalho nos casos comuns, e facilitam a classificação das exceções nos demais casos. Agora, **por que você iria querer gerar as suas próprias exceções?** Vejamos:

+ Identificar problemas específicos

O Java possui várias classes de exceções pré-definidas que podem lidar com muitos problemas comuns (por exemplo, **NullPointerException**, **IndexOutOfBoundsException**). No entanto, essas classes de exceções não podem abranger todos os possíveis problemas que podem ocorrer em um programa. Por isso, você pode definir condições de erro específicas para sua aplicação ao criar as suas próprias exceções.

+ Fornecer feedback significativo

Exceções personalizadas podem fornecer mensagens de erro muito mais informativas e específicas do que as exceções padrão do Java. Isso pode ser extremamente útil para o processo de debug do seu próprio código, pois pode ajudar a identificar rapidamente onde e por que um erro ocorreu.

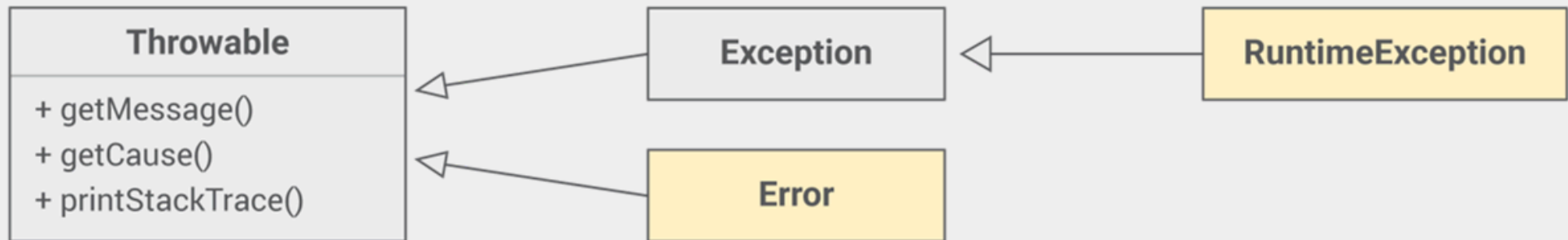
+ Controlar o fluxo do programa

Você também pode querer interromper a execução normal do programa quando uma condição de erro ocorre e transferir o controle para um bloco **catch** adequado. Isso é útil em situações em que continuar a execução normal do programa pode levar a comportamentos indesejados ou a erros ainda mais graves.

Dito isso, vamos entender um pouco mais sobre as exceções.

Tipos de exceções

Primeiramente, é importante saber que existem diferentes **tipos de exceções**. A hierarquia de classes das exceções começa com quatro classes importantes.



Fonte: Autores (2021).

A classe throwable representa qualquer coisa que possa ser disparada como exceção e capturada na cláusula catch. Destacamos três métodos importantes:

- **getMessage**: observe que o Java especifica que todas as exceções devem possuir uma mensagem descritiva de erro. Mesmo quando criamos nossas exceções, devemos nos preocupar em escrever um texto descritivo do problema ocorrido.
- **getCause**: em algumas situações, pode ser interessante capturar uma exceção específica e gerar outra mais abrangente. Por exemplo, em um método que faz a carga dos dados da sua aplicação de diferentes locais, você pode querer capturar a exceção específica das várias fontes de dados (SQLException, IOException etc.) e disparar uma exceção mais geral. Nesse caso, é possível incluir a exceção original como causadora.
- **printStackTrace**: imprime o **stack trace** da exceção.

Vamos testar rapidamente estes três?



EXEMPLO

Exemplo de aplicação 6: crie um código em Java contendo uma classe chamada **Exemplo.java**.

Cole o código a seguir e teste o seu código. Lembre-se que a divisão de números inteiros em Java ignora o resto.

</>

```
1 public class Exemplo {
2     static int realizarConta(int x, int y) {
3         return y / x;
4     }
5
6     public static void main(String[] args) {
7         try {
8             int z = realizarConta(0, 10);
9             System.out.println(z);
10        } catch (ArithmeticException e) {
11            System.out.println("Mostrando o getMessage(): ");
12            System.out.println(e.getMessage());
13
14            System.out.println("\nMostrando o getCause(): ");
15            System.out.println(e.getCause());
16
17            System.out.println("\nMostrando o printStackTrace(): ");
18            e.printStackTrace();
19        }
```

Ao executar o código você verá um resultado como este:


```
Mostrando o getMessage():  
/ by zero  
  
Mostrando o getCause():  
null  
  
Mostrando o printStackTrace():  
java.lang.ArithmeticException: / by zero  
    at Main.realizarConta(Main.java:3)  
    at Main.main(Main.java:8)  
  
Process finished with exit code 0
```

Veja que o resultado de **getMessage()** mostra somente a causa do erro (a divisão por zero). Já **getCause()** retorna nulo: afinal, o `ArithmeticException` não foi causado por outra exceção. Finalmente, **printStackTrace()** mostra na tela o caminho completo do erro.

Entretanto, quando formos criar nossas próprias exceções, jamais as criaremos como filhas diretas da classe *Throwable*. No lugar, o Java fornece as classes *Exception* e *RuntimeException* para isso. Elas representam os dois tipos de exceção da linguagem:

+ Exceções verificadas (filhas de *Exception*)

Devem ser obrigatoriamente capturadas pelo programador. No caso delas, a cláusula *try...catch* será obrigatória, ou o método terá de sinalizar que dispara aquela exceção. Geralmente representam problemas comuns, relacionados à situação sendo modelada. Por exemplo, as classes que carregam arquivos no Java disparam uma exceção verificada do tipo `IOException`. Isso ocorre porque há vários problemas comuns na carga de arquivos (arquivo não existir, estar corrompido, não ter permissão de leitura etc.).

+ Exceções não verificadas (filhas de *RuntimeException*)

Não precisam ser tratadas pelo programador. Geralmente, representam problemas de programação, que poderiam ser prevenidos de outra forma. A exceção que vimos no exemplo, *ArithmeticException*, é um tipo de exceção desse tipo.

E a classe *Error*? Ela também irá disparar exceções não verificadas, porém, representam erros graves, cujo tratamento não é possível. Frequentemente, ela só é capturada para fins de registro. Elas geralmente são disparadas pela Virtual Machine (VM). Um exemplo de erro é a falta de memória. Não há o que a aplicação fazer caso um *OutOfMemoryError* seja disparado.

Disparando exceções não verificadas

Para disparar uma exceção, utilizamos a cláusula ***throw***, com o objeto da exceção a ser disparado. O Java possui uma série de classes-padrão de exceção para problemas comuns, veja alguns exemplos na tabela a seguir.

Exceção	Verificada?	Descrição
CloneNotSupportedException	Sim	O método clone() não pode ser chamado nesse objeto.
IllegalArgumentException	Não	O parâmetro de um método possui valor inválido.
IllegalStateException	Não	Um método foi chamado em um momento indevido.
IndexOutOfBoundsException e ArrayIndexOutOfBoundsException	Não	O sistema tentou acessar um índice inválido em uma lista ou array.

IOException	Sim	Problema na leitura/escrita de arquivos.
NullPointerException	Não	Um método foi chamado em uma variável nula.
SQLException	Sim	Problema reportado pelo banco de dados.



EXEMPLO

Exemplo de aplicação 7: crie um código em Java contendo uma classe chamada **Pessoa.java**. O arquivo deve possuir o código informado a seguir.

</>

```
1 public class Pessoa {
2     private int idade;
3     private String nome;
4
5     public Pessoa(int idade, String nome){
6         setIdade(idade);
7         this.nome = nome;
8     }
9
10    public void setIdade(int idade) {
11        if (idade <= 0 || idade >= 130) {
12            throw new IllegalArgumentException("Idade inválida: " + idade);
13        }
14        this.idade = idade;
15    }
16
17    public static void main(String[] args) {
18        System.out.println("Tentando criar uma pessoa com 20 anos...");
19        Pessoa ze = new Pessoa(20, "Zé");
```

Veja que criamos o método `setIdade`, da classe `Pessoa`. Sabemos que pessoas não possuem idades negativas e não podem viver centenas de anos. Então, poderíamos implementar o método conforme o exemplo apresentado.

Agora, mande compilar o código (build). Deu certo, não é? Não tivemos exceções até agora.

Agora, mante executar o código (run). Veja que apareceu isto como resultado:

```
Tentando criar uma pessoa com 20 anos...
Tentando criar uma pessoa com -10 anos...
Exception in thread "main" java.lang.IllegalArgumentException: Idade inválida: -10
```

```
at Pessoa.setIdade(Pessoa.java:12)
at Pessoa.(Pessoa.java:6)
at Pessoa.main(Pessoa.java:22)
```

Process finished with exit code 1

Vamos entender o que houve:

1. Conseguimos criar um objeto para uma pessoa com 20 anos (linha 19).
2. Depois, tivemos uma exceção ao tentar cadastrar uma pessoa com -10 anos (linha 22).
 - a. Veja o stack trace do erro, e leia de baixo para cima:
 - i. **at Pessoa.main(Pessoa.java:22)**: o erro começou na linha 22 (ao tentar cadastrar o João)
 - ii. **at Pessoa.(Pessoa.java:6)**: para cadastrar uma nova pessoa, o código foi da linha 22 para a linha 6. Isto é: entramos no construtor e, do construtor, fomos para o método **setIdade**. Aqui, o valor de **idade** é -10: um valor negativo.
 - iii. **at Pessoa.setIdade(Pessoa.java:12)**: como o teste do **if** da linha 11 é verdadeiro (-10 é menor do que 0), a linha 12 foi executada.
 1. Linha 12 (**throw new IllegalArgumentException("Idade inválida: " + idade)**): esta linha de código (linha 12) significa que estamos **gerando uma exceção** do tipo `IllegalArgumentException` e informando que o argumento ilegal foi a idade.
 2. No caso, estamos avisando que não podemos trabalhar com idades que sejam menores ou iguais a zero ou, ainda, maiores do que 30.
 3. Por isso que apareceu o erro “**Idade inválida: -10**” na saída do nosso código.

Testar parâmetros dessa forma é uma boa prática, e tem até um nome: **código de guarda** (code guard).



IMPORTANTE

Procure sempre programar de maneira preventiva, isso evita que um erro (como uma idade negativa) se propague pelo seu código. acredite: vai ser bem mais fácil diagnosticar e corrigir um problema se uma exceção for disparada no ponto exato em que ele foi gerado, do que o descobrir dias depois, pois há valores errados em seu banco de dados!

Disparando exceções verificadas

Algumas exceções sinalizam problemas legítimos, que deveriam ser constantemente verificados pelo programador. Por exemplo, numa classe `ContaCorrente`, poderíamos querer sinalizar a possibilidade de não haver saldo no momento do saque – situação que tornaria o saque impossível.

Para esses casos, utilizamos as **exceções verificadas**, ou seja, àquelas que são filhas de *exception*, mas não de *RuntimeException*. Embora não seja uma regra absoluta, é comum também que as exceções verificadas sejam de tipos criados por nós, já que elas validam regras de negócio mais específicas, e não problemas comuns de programação.

Como implementaríamos, então, o método sacar? Veja o exemplo a seguir.



EXEMPLO

Exemplo de aplicação 8: crie um projeto em Java contendo dois arquivos. Um deles se chamará **`SaldoInsuficienteException.java`**. O outro se chamará **`ContaCorrente.java`**.

Agora, cole o código a seguir dentro do **`SaldoInsuficienteException.java`**.

</>

```
1 public class SaldoInsuficienteException extends Exception {  
2     public SaldoInsuficienteException(String msg) {  
3         super(msg);  
4     }  
5 }
```

Agora, cole o código a seguir dentro do **ContaCorrente.java**.

</>

```
1 public class ContaCorrente {
2     private double saldo;
3
4     public ContaCorrente(double saldo){
5         this.saldo = saldo;
6     }
7
8     public void sacar(double valor) throws SaldoInsuficienteException {
9         if (saldo - valor <= 0) {
10             throw new SaldoInsuficienteException(String.format(
11                 "O saldo %.2f é insuficiente para sacar o valor %.2f",
12                 saldo, valor)
13             );
14         }
15         this.saldo -= valor;
16     }
17
18     public static void main(String[] args) {
19         ContaCorrente conta = new ContaCorrente(100);
```

Observe que, inicialmente, fizemos a criação da classe `SaldoInsuficienteException`, classe-filha de exception. Fizemos a chamada ao construtor da classe-pai para repassar a mensagem de erro (linha 3 em **`SaldoInsuficienteException.java`**).

Na classe `ContaCorrente`, iniciamos com o código de guarda, como fizemos no método `setIdade()`, testando se o saldo seria suficiente para o saque (linhas 9 a 14 em **`ContaCorrente.java`**). A ideia desse código é evitar saques se a pessoa não possui dinheiro suficiente na conta. Se o saldo não for suficiente, criamos a exceção e a disparamos com o `throw`. Porém, observe que tomamos o cuidado de gerar uma mensagem bastante descritiva (linhas 11 e 12). Isso nos auxiliará a entender em detalhes em que situação o problema ocorreu, caso ele venha a acontecer.

Observe também que há mais um detalhe nesse código: na declaração do método sacar, na linha 8, agora encontramos a cláusula `throws` e o nome da exceção. Isso indica que esse método dispara essa exceção verificada e que ela deve ser **obrigatoriamente** tratada. Nesse caso, o programador que chamar o método sacar tem duas opções:

- Adicionar um bloco *try...catch* que capture e trate a exceção, impedindo que ela se propague. Foi o que fizemos nas linhas 21 a 25.
- Indicar que o método que usa o método sacar não trata essa exceção e, portanto, também pode dispará-la. Nesse caso, esse método também terá que conter a cláusula `throws`.

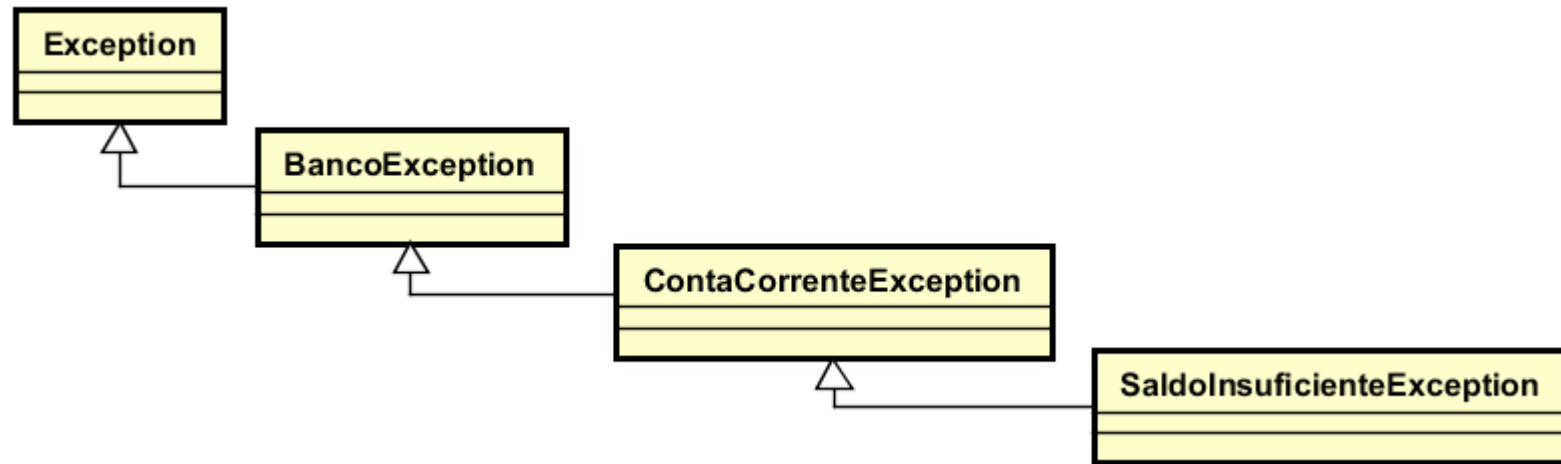


IMPORTANTE

Observe que, se uma nova exceção verificada for adicionada a um método (parecido com o que fizemos na linha 8 com o `throws`), o Java exigirá que **todos** os pontos que chamam aquele método sejam modificados.

Felizmente, a cláusula **throws** também é hierárquica. Por isso, é interessante criarmos uma **hierarquia de exceções** em nosso sistema. Por exemplo, a exceção `SaldoInsuficienteException` poderia ser filha de `ContaCorrenteException`, que por sua vez poderia ser filha de `BancoException` – essa sim, mãe de todas as exceções de nosso sistema. Poderíamos então reorganizar o método sacar para ter em sua cláusula **throws** uma `ContaCorrenteException`, mesmo que seu `throw` interno ainda dispare a exceção específica.

Vejamos o exemplo a seguir. Aqui, `ContaCorrenteException` possui classes-filhas como `SaldoInsuficienteException` e outros casos possíveis (e fictícios) como `LimiteInsuficienteException`, `ContaEncerradaException`. Veja também a adição do `throws` na linha 8 e do `throw new` na linha 10.



</>

```
1 public class ContaCorrente {
2     private double saldo;
3
4     public void sacar(double valor) throws ContaCorrenteException {
5         if (saldo - valor < 0) {
6             throw new SaldoInsuficienteException(String.format(
7                 "O saldo %.2f é insuficiente para sacar o valor %.2f",
8                 saldo, valor)
9             );
10        }
11        this.saldo -= valor;
12    }
13 }
```

Observe que isso faz bastante sentido. Normalmente, quem faz **catch** no método sacar irá querer pegar qualquer problema relacionado ao saque (saldo insuficiente, limite insuficiente, conta encerrada etc.). Ou seja, estamos indicando na cláusula **throws** que o `ContaCorrenteException` é o nível ideal para a captura. Porém, ainda fornecemos exceções mais específicas, para caso seja necessário tratar um dos problemas maneira especial. Da mesma forma, lembre-se: o programador que utiliza o método **sacar()** ainda tem a liberdade de capturar uma exceção ainda mais alta (como `BancoException`) caso queira, pois a cláusula `catch` também é hierárquica.



DICA

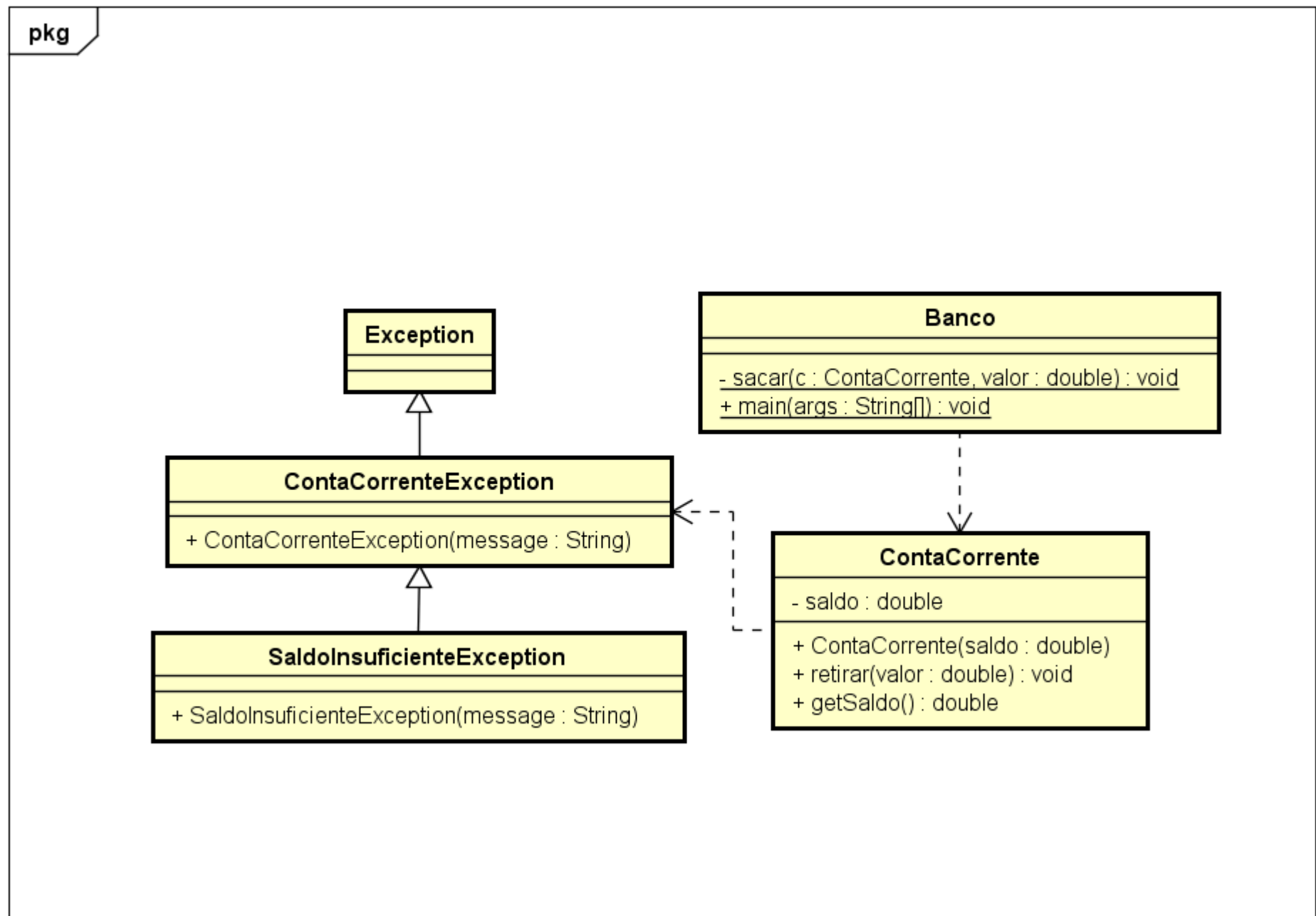
Organize cuidadosamente a hierarquia de classes de exceção do seu sistema.



EXEMPLO



Exemplo de aplicação 9: crie um projeto em Java reproduzindo o diagrama UML, a seguir. Veja que se trata de um diagrama de classes do programa para tratamento de exceção verificada, o qual está em uma estrutura hierárquica.



Tente implementar por conta própria. De qualquer modo, veja aqui um exemplo de implementação:

ContaCorrenteException.java:

</>

```
1 // Exceção mais genérica
2 public class ContaCorrenteException extends Exception {
3     public ContaCorrenteException(String message) {
4         super(message);
5     }
6 }
```

SaldoInsuficiente.java:

</>

```
1 // Exceção mais específica
2 public class SaldoInsuficienteException extends ContaCorrenteException {
3     public SaldoInsuficienteException(String message) {
4         super(message);
5     }
6 }
```

ContaCorrente.java:

</>

```
1 public class ContaCorrente {
2     private double saldo;
3     public ContaCorrente(double saldo) {
4         this.saldo = saldo;
5     }
6     // Declaração do o método retirar, que pode lançar uma exceção
7     // mais genérica: ContaCorrenteException
8     public void retirar(double valor) throws ContaCorrenteException {
9         // Em erro específico, lança exceção específica:
10        // SaldoInsuficienteException, filha de ContaCorrenteException
11        if (saldo - valor <= 0)
12            throw new SaldoInsuficienteException(String.format(
13                "O saldo R$ %.2f é insuficiente para sacar o valor R$ %.2f",
14                saldo, valor)
15            );
16        else
17            this.saldo -= valor;
18    }
19    public double getSaldo() {
```

Banco.java:

</>

```
1 public class Banco {
2     // Método sacar DELEGA o tratamento de exceção genérica
3     // (ContaCorrenteException), pois não fará o seu tratamento
4     private static void sacar (ContaCorrente c, double valor) throws
5         ContaCorrenteException {
6         c.retirar(valor);
7     }
8
9     public static void main(String[] args) {
10         ContaCorrente cta = new ContaCorrente(1000);
11
12         try {
13             sacar(cta,200);
14         } catch (ContaCorrenteException e) {
15             System.out.println("Erro: " + e.getMessage());
16         }finally {
17             System.out.println("Saldo Conta: R$ " + cta.getSaldo());
18         }
19     }
```

Agora, execute o programa a partir de **Banco.java**. Testamos um caso em que tínhamos uma conta com R\$ 800 e tentamos (sem sucesso) sacar R\$ 2000. Vamos entender melhor a implementação com algumas perguntas:

1. Por que o método `retirar()` da classe `ContaCorrente` está declarado para lançar (**throws**) a exceção `ContaCorrenteException`, contudo em caso de erro, lança de fato (**throw new**) a exceção `SaldoInsuficienteException`?

a. Como **`SaldoInsuficienteException`** é herdeira de **`ContaCorrenteException`**, a declaração do método **`retirar()`** indica que ele pode lançar uma exceção mais genérica: **`ContaCorrenteException`**. Contudo, quando há um erro específico, o método pode lançar uma exceção específica, herdeira de **`ContaCorrenteException`**, como é o caso da exceção **`SaldoInsuficienteException`**.

2. Se o método `retirar()` da classe `ContaCorrente` executar o comando “`throw new SaldoInsuficienteException();`” qual será o valor do saldo da conta corrente após esse comando?
- a. Se houver o lançamento da exceção, a retirada não ocorre e o saldo mantém o valor.
3. Na classe `banco`, na execução da **linha 13**, foi gerada exceção? Por quê?
- a. Não foi gerada nem lançada exceção, pois o saldo (R\$1000,00) era suficiente para a retirada solicitada (R\$200,00).
4. Na classe `banco`, **linha 23**, de que classe é a instância de exceção “e” tratada?
- a. Mesmo sendo `catch (ContaCorrenteException)`, a exceção tratada de fato é a `SaldoInsuficienteException`, herdeira de `ContaCorrenteException`, lançada pelo método `retirar()`.
5. No método `main` da classe `banco`, seria possível retirar os comando `try-catch`? Por quê?
- a. Não, pois tanto o método `retirar()` quanto o `sacar()` lançam exceção verificada em tempo de compilação (gera erro de compilação). Ou seja, impõem a necessidade de tratamento com `try-catch`.
6. No método `main` da classe `banco`, os comando `try-catch` envolvem o método `sacar()`. Por quê?
- a. Porque o método `sacar()` chama o método `retirar()`, o que obriga a `sacar()` a tratar (`try-catch`) ou a delegar (`throw`) a exceção lançada por `retirar()`.
7. Na classe `banco`, qual o valor apresentado pela execução do bloco `finally` da **linha 24**? Por que esse valor é apresentado?
- a. O valor é de R\$ 800,00, que é o valor anterior à operação de `retirar()`, não é executada, uma vez que o saldo é insuficiente para uma retirada de R\$ 2.000,00.

| Testes unitários

Obviamente, queremos que nossos sistemas sejam robustos e livres de erros. Podemos utilizar o mecanismo de captura de erros a nosso favor, testando por potenciais problemas ou validando situações em vários pontos do código. De fato, há ferramentas inteiras dedicadas aos testes de sistemas, como o **JUnit**. A linguagem Java fornece uma instrução simples com o este objetivo, chamada `assert`.

O comando `assert` também testará uma condição e **disparará uma exceção**, similar ao que ocorre com os códigos que geramos até agora. Porém, há uma diferença: ele só fará isso caso passemos um comando específico para a JVM na hora de executar nossa aplicação. Veja como ativar as exceções do `assert`, habilitando essa verificação na JVM (no Eclipse e no IntelliJ).



IMPORTANTE

O uso de **`assert`** nativo do Java (que exige ativar a flag `-ea` na JVM) é extremamente raro em desenvolvimento comercial moderno. Empresas com códigos mais novos usam frameworks de teste (JUnit, TestNG) ou validações explícitas (`if/throw`). Contudo, no mercado de trabalho é comum termos que manter códigos mais antigos (também chamados de “códigos legados”). Além disso, outras linguagens de programação que usam POO também usam isto atualmente. É por isso que também mostramos o **`assert`** para você

Comando **`assert`**: apenas dispara exceção caso passemos um parâmetro específico na hora de executar nossa aplicação na **Virtual Machine (VM)**.

- **No Eclipse:** botão direito sobre classe à Run As à Run Configurations... à Arguments à VM Arguments: `-ea`

- **No IntelliJ:** Run → Edit Configurations → Add New Configuration → Application → Modify options → VM options: -ea → escolha a classe principal do projeto.

Então, para que utilizamos? Para validar pressupostos de programação, ou seja, para escrever testes que validem nosso próprio código (**testes unitários**), em uma destas situações:

1. **Pré-condições:** situações que devem ser verdadeiras quando o método foi invocado.
2. **Pós-condições:** situações que devem ser verdadeiras quando o método foi finalizado.
3. **Invariantes:** situações que permanecem inalteradas em todo programa. Geralmente, testada ao fim de métodos públicos.

Que tal observarmos um exemplo prático? Você não precisará executar o código a seguir, mas gostaria que você o analisasse – em especial, as situações de pré-condições, pós-condições e invariantes:

</>

```
1 class CarrinhoDeCompras {
2     private static int maximo = 10;
3     private static int quantidade = 0;
4     private static int inseridos = 0;
5     private static int removidos = 0;
6     private static double preco_unitario = 10.00;
7
8     public static void inserir() {
9         assert (quantidade < maximo); // PRÉ-CONDIÇÃO
10        quantidade++;
11        inseridos++;
12        assert (quantidade == inseridos - removidos); // INVARIANTE
13    }
14
15    public static void remover() {
16        assert (quantidade > 0); // PRÉ-CONDIÇÃO
17        quantidade--;
18        removidos++;
19        assert (quantidade >= 0); // PÓS-CONDIÇÃO
```

Este código possui as seguintes asserções:

- **Pré-condições:** na inserção, o carrinho não deveria ter mais itens do que a quantidade máxima. Na remoção, há a necessidade de haver pelo menos um item.
- **Pós-condições:** o método remover jamais poderá deixar o carrinho com uma quantidade negativa de itens. Isso só ocorrerá na presença de um erro de programação. Por isso, deixamos esse pressuposto explícito.
- **Invariantes:** observe que indiferentemente de inserimos ou removermos, a quantidade total de itens do carrinho precisará ser igual à quantidade de itens inseridos subtraído dos itens removidos. Testes de invariantes são bastante úteis quando testamos redundâncias de informação em nossas classes, ou seja, informações que podem ser obtidas de duas formas diferentes – como no exemplo.

| Assertões e exceções

Você pode estar se perguntando: no caso das pré-condições, não seria melhor utilizar exceções no lugar das pré-condições?

A resposta é: depende. Quando analisar se deve ou não utilizar uma asserção, questione:

1. O que está sendo testado é uma regra de negócio? Ou você está se prevenindo como programador de um erro seu? Se for o primeiro caso, prefira exceções.
2. O que está sendo testado é uma situação comum? Ou é algo que só ocorrerá no caso de bugs? Se for o primeiro caso, prefira exceções.
3. Se você remover o teste, valores incorretos poderão ser inseridos no sistema? Se a resposta for positiva, prefira exceções.

Refletindo a respeito, o que você deduz sobre o carrinho de compras? Sim: o sistema estaria mais bem modelado com exceções e códigos de guarda nessas condições.

Mas voltemos ao caso do método *setIdade()*, visto no **exemplo de aplicação 7**. Obviamente, uma idade negativa é um problema, mas será que deveríamos testar a idade máxima com tanto rigor? Podemos usar uma **asserção para sinalizar uma situação estranha**, enquanto mantemos o teste para o que certamente é o fruto de um problema.

Assim, aquele código poderia ficar conforme o exemplo a seguir. Preste atenção na linha 12:

```
</>
```

```
1 public class Pessoa {
2     private int idade;
3     private String nome;
4
5     public Pessoa(int idade, String nome){
6         setIdade(idade);
7         this.nome = nome;
8     }
9
10    public void setIdade(int idade) {
11        //PRÉ-CONDIÇÃO
12        assert(idade < 95);
13
14        if (idade <= 0 || idade >= 130) {
15            throw new IllegalArgumentException("Idade inválida: " + idade);
16        }
17        this.idade = idade;
18    }
19 }
```

Observe que a exceção se manteve, mas com um parâmetro **menos** rigoroso do que a asserção. Afinal, a pessoa mais velha do mundo com comprovação científica teve 122 anos de idade – mais do que isso, obviamente, é um erro de cadastro. Algo entre 95 e 130 pode até ser uma situação possível, embora pouco provável. Se não há uma regra de negócios explicitamente mapeada indo contra essa situação, esse cadastro **deveria** ser possível.

Por outro lado, a asserção está testando um pré-condição que o time de programação considerou quando elaborou esse sistema: tal situação é tão improvável que eles acham que não irá acontecer. Isso pode ter implicações importantes, como uma caixa de textos do sistema não ser larga o suficiente para um usuário com três dígitos em sua idade, ou mesmo não ter uma opção para data tão antiga caso um calendário seja apresentado.

Isso permitirá que o sistema sinalize um problema no desenvolvimento, se futuramente essa regra for violada.



EXEMPLO

Exemplo de aplicação 10: crie um projeto em Java contendo informações de um carrinho de compras de um e-commerce conforme o código a seguir.

Arquivo Carrinho.java (dentro do package carrinho):

</>

```
1 package carrinho;
2
3 public class Carrinho {
4     private int    qtdMaxima;
5     private int    quantidade = 0;
6     private int    inseridos = 0;
7     private int    removidos = 0;
8     private double  precoUnitario;
9
10    public Carrinho (int qtdMaxima, double precoUnitario) {
11        this.qtdMaxima = qtdMaxima;
12        this.precoUnitario = precoUnitario;
13    }
14    public void printCarrinho() {
15        System.out.println(" - Qtd. máxima de itens no carrinho = " + qtdMaxima);
16        System.out.printf (" - Preço unitário de cada item      = R$ %.2f\n", precoUnitario);
17    }
18
19    public void inserir    () {
```

Arquivo Comprador.java (dentro do package carrinho):

</>


```
1 package carrinho;
2
3 public class Comprador {
4     private String nome;
5     private double valorTotalCarteira;
6     private Carrinho carrinho;
7
8     public Comprador(String nome, double valorTotalCarteira, int qtdCarrinho, double precoUnitario) {
9         this.nome = nome;
10        this.valorTotalCarteira = valorTotalCarteira;
11        this.carrinho = new Carrinho(qtdCarrinho, precoUnitario);
12    }
13    public double getValorTotalCarteira() {
14        return valorTotalCarteira;
15    }
16    public void printComprador() {
17        System.out.println("Nome comprador(a): " + nome);
18        System.out.printf ("Valor para gastar: R$ %.2f\n",
19                            valorTotalCarteira);
```

Agora, execute o código a partir de **Comprador.java**. O programa original **não aponta erro**, pois a **JVM não foi habilitada** para pegar erros de assert: observe o valor a pagar negativo e o total de itens no carrinho também negativo:

```
Console  #ParaTodosVerem 
Nome comprador(a): Maria
Valor para gastar: R$ 10,00
- Qtd. máxima de itens no carrinho = 5
- Preço unitário de cada item      = R$ 15,00
-----
Itens no carrinho, após inclusão = 6
Itens no carrinho, após remoção  = -2
Valor a pagar: R$ -30,00
```

Por fim, **habilite a JVM** para pegar erros de assert. Agora é possível verificar os erros pegos pelo assert: veja que tentamos incluir seis itens, contudo o carrinho comporta apenas cinco.



DICA

Dica: para habilitar a JVM para pegar erros de assert, faça:

No Eclipse: botão direito sobre classe → Run As → Run Configurations... → Arguments → VM Arguments: -ea

No IntelliJ: Run → Edit Configurations → Add New Configuration → Application → Modify options → VM options: -ea → escolha a classe principal do projeto.

```
Console #ParaTodosVerem
Nome comprador(a): Maria
Valor para gastar: R$ 10,00
- Qtd. máxima de itens no carrinho = 5
- Preço unitário de cada item      = R$ 15,00
-----
Erro - Inclusão: carrinho cheio, já com 5 itens.
```

Finalmente, **altere o código do programa** para obter os erros de:

- **Carrinho vazio**, quanto tentamos remover itens de um carrinho vazio.

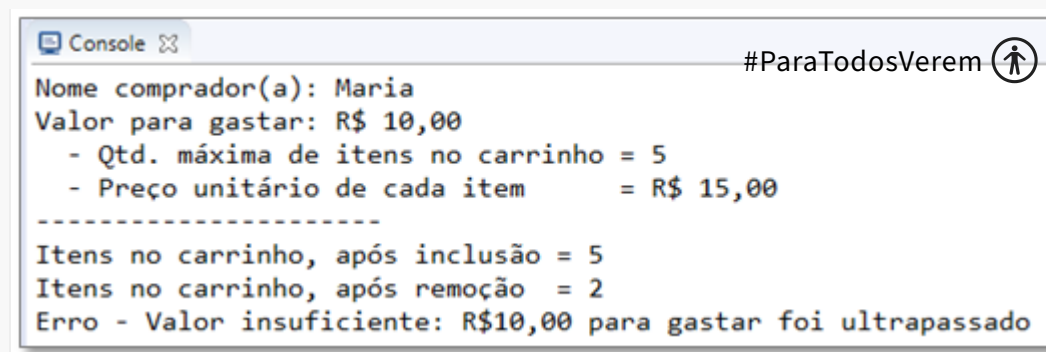
```
Console #ParaTodosVerem
Nome comprador(a): Maria
Valor para gastar: R$ 10,00
- Qtd. máxima de itens no carrinho = 5
- Preço unitário de cada item      = R$ 15,00
-----
Itens no carrinho, após inclusão = 4
Erro - Remoção: carrinho vazio, não é possível retirar mais itens.
```

Em **Comprador.java**, faça:

</>

```
1 package carrinho;
2
3 public class Comprador {
4
5     int totalInclusao = 4;
6
7 }
```

- **Valor insuficiente**, quando não há valor em carteira suficiente para pagar pelos itens do carrinho.



A screenshot of a Java console window. The title bar says 'Console' with a maximize icon. In the top right corner, there is a hashtag '#ParaTodosVerem' followed by a person icon. The console output is as follows:

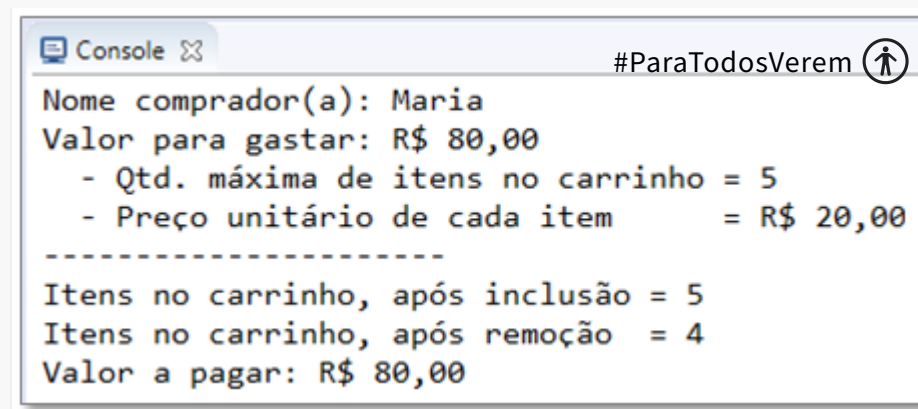
```
Nome comprador(a): Maria
Valor para gastar: R$ 10,00
- Qtd. máxima de itens no carrinho = 5
- Preço unitário de cada item      = R$ 15,00
-----
Itens no carrinho, após inclusão = 5
Itens no carrinho, após remoção  = 2
Erro - Valor insuficiente: R$10,00 para gastar foi ultrapassado
```

Em **Comprador.java**, faça:

</>

```
1 package carrinho;
2
3 public class Comprador {
4
5     int totalInclusao = 5;
6     int totalRemocao = 3;
7
8 }
```

- **Compra bem-sucedida**, quando todas as restrições são atendidas e Maria consegue levar os itens comprados no carrinho.



The screenshot shows a console window titled "Console" with a close button. The output text is as follows:

```
#ParaTodosVerem ⓘ
Nome comprador(a): Maria
Valor para gastar: R$ 80,00
  - Qtd. máxima de itens no carrinho = 5
  - Preço unitário de cada item      = R$ 20,00
-----
Itens no carrinho, após inclusão = 5
Itens no carrinho, após remoção  = 4
Valor a pagar: R$ 80,00
```

</>

```
1 package carrinho;
2
3 public class Comprador {
4
5     int totalInclusao = 5;
6     int totalRemocao = 1;
7
8     Comprador maria = new Comprador ("Maria", 80.0, 5, 20.0);
9
10 }
```

Exercícios de fixação



EXERCÍCIO

1. Imagine que você está criando um recurso de upload de vídeo para um aplicativo semelhante ao TikTok. Crie uma classe **UploadDeVideo** com um método **upload()**. Esse método deve simular o lançamento de exceções como **FileTooLargeException** e **NetworkErrorException**. Use um bloco **try-catch** para lidar com essas exceções.
2. Suponha que você está trabalhando em um sistema de pagamento para um site de e-commerce como o AliExpress. Crie um método **processarPagamento()** que pode lançar exceções como **InsufficientFundsException** ou **InvalidPaymentMethodException**. Use um bloco **try-catch-finally** para garantir que, independentemente de ocorrer uma exceção, uma mensagem de finalização seja impressa para indicar que a tentativa de processamento de pagamento foi concluída.
3. Ao desenvolver o software de controle para um micro-ondas da Electrolux, você precisa garantir que a porta do aparelho esteja fechada antes que ele comece a funcionar. Implemente isso simulando o lançamento de uma exceção **DoorNotClosedException** se a porta não estiver fechada, e use **try-catch** para lidar com essa exceção.
4. Você está desenvolvendo um sistema para máquinas de venda automática da Coca-Cola que aceita várias formas de pagamento (notas, moedas, cartão). Crie exceções diferentes para cenários como **NotaInvalidaException**,

MoedaInvalidaException, **ErroLeituraCartaoException** e use um bloco **try-catch** com multi-catch para lidar com essas exceções.

5. Enquanto desenvolve o sistema de matrículas da PUCPR, crie um método **matricularAluno()** que possa lançar exceções como **AlunoJaMatriculadoException** e **CursoLotadoException**. Demonstre a captura hierárquica usando **try-catch** blocos para tratar essas exceções e suas possíveis subclasses.



Resolução do exercício



Exercício 1

</>

```
1 class FileTooLargeException extends Exception {
2     public FileTooLargeException(String errorMessage) {
3         super(errorMessage);
4     }
5 }
6
7 class NetworkErrorException extends Exception {
8     public NetworkErrorException(String errorMessage) {
9         super(errorMessage);
10    }
11 }
12
13 class UploadDeVideo {
14     void upload(String videoFilePath) throws FileTooLargeException, NetworkErrorException {
15         // Simulando a verificação do tamanho do arquivo de vídeo
16         double videoFileSize = Math.random() * 1024; // Simula o tamanho do arquivo de vídeo
17         if (videoFileSize > 500) { // Vamos assumir que 500 é o tamanho máximo permitido par
18             throw new FileTooLargeException("O arquivo de vídeo é muito grande!");
19         }
20     }
21 }
```

Exercício 2

</>


```
1 class InsufficientFundsException extends Exception {
2     public InsufficientFundsException(String errorMessage) {
3         super(errorMessage);
4     }
5 }
6
7 class InvalidPaymentMethodException extends Exception {
8     public InvalidPaymentMethodException(String errorMessage) {
9         super(errorMessage);
10    }
11 }
12
13 class Pagamento {
14     void processarPagamento(String metodoDePagamento, double valor) throws InsufficientFunds
15         double saldo = 100; // Vamos supor que o saldo seja 100 para este exemplo
16         if (valor > saldo) {
17             throw new InsufficientFundsException("Saldo insuficiente para realizar a compra.
18         }
19 }
```

Exercício 3

</>

```
1 class DoorNotClosedException extends Exception {
2     public DoorNotClosedException(String errorMessage) {
3         super(errorMessage);
4     }
5 }
6
7 class Microondas {
8     boolean portaAberta;
9
10    void iniciar() throws DoorNotClosedException {
11        if (portaAberta) {
12            throw new DoorNotClosedException("A porta do micro-ondas está aberta.");
13        }
14
15        // Se a porta estiver fechada, o micro-ondas é "iniciado"
16        System.out.println("Micro-ondas iniciado com sucesso!");
17    }
18
19    public static void main(String[] args) {
```

Exercício 4

</>

```
1 class NotaInvalidaException extends Exception {
2     public NotaInvalidaException(String errorMessage) {
3         super(errorMessage);
4     }
5 }
6
7 class MoedaInvalidaException extends Exception {
8     public MoedaInvalidaException(String errorMessage) {
9         super(errorMessage);
10    }
11 }
12
13 class ErroLeituraCartaoException extends Exception {
14     public ErroLeituraCartaoException(String errorMessage) {
15         super(errorMessage);
16     }
17 }
18
19 class MaquinaVendaAutomatica {
```

Exercício 5

</>

```
1 class AlunoJaMatriculadoException extends Exception {
2     public AlunoJaMatriculadoException(String errorMessage) {
3         super(errorMessage);
4     }
5 }
6
7 class CursoLotadoException extends Exception {
8     public CursoLotadoException(String errorMessage) {
9         super(errorMessage);
10    }
11 }
12
13 class SistemaMatriculasPUCPR {
14     void matricularAluno(String aluno, String curso) throws AlunoJaMatriculadoException, Cur
15         // Aqui, inseriríamos as regras de negócio para verificar se o aluno já está matricu
16         // Neste exemplo, estou jogando as exceções diretamente para simplificar
17
18         if ("aluno1".equals(aluno)) {
19             throw new AlunoJaMatriculadoExcention("O aluno iá está matriculado.");
```

Referências

GODOY, V. **Programação orientada a objetos I**. Curitiba: IESDE, 2019.

HORSTMANN, C. S.; CORNELL, G. **Core Java – Volume I**. 8. ed. São Paulo: Pearson, 2010.

